

MASTER PROJECT REPORT AND DEVELOPMENT SPECIFICATION

Problem 10 — Autonomous Personal Finance Optimization & Explainable Micro-Advisory Engine

Working product name: Financial Decision Replay Engine

Signature feature name: Financial Time Machine

Domain: FinTech, Applied Machine Learning, Explainable AI, Financial Data Engineering, and Mathematical Optimization

Primary currency: INR (₹)

Target: A working, demonstrable, hackathon-ready full-stack application

1. INSTRUCTIONS TO THE CODING AI

Act as a senior full-stack developer, Python engineer, data scientist, machine-learning engineer, database architect, mathematical optimization specialist, UI/UX designer, security engineer, and software tester.

Build the complete application described in this document.

Do not stop after generating a plan, a static UI, or sample code. Implement the frontend, backend, database, transaction-processing pipeline, financial calculations, forecasting, savings optimizer, report viewer, evidence traceability, and testing.

Before coding:

1. Inspect the existing repository and identify the current technology stack.
2. Review the files, dependencies, database setup, existing routes, and implemented functionality.
3. Identify what already works and what is missing.
4. Create a concise implementation plan based on the existing code.
5. Establish a prioritized build sequence.
6. Begin implementation after the plan, without waiting for unnecessary confirmation.

Preserve useful existing code. Do not replace a functioning project with a new scaffold without a good technical reason.

Follow clean-code principles throughout. Use modular architecture, reusable components, clear naming, type checking, validation, meaningful error handling, and automated tests.

Do not make every feature look complete when its backend is not implemented. Every important button, form, chart, connection status, report explanation, and scenario action must work.

If a real third-party integration is unavailable, implement a clearly labelled alternative rather than a fake integration.

At the end, explain what is implemented, what has been tested, what remains incomplete, and exactly how to run the project.

2. PROBLEM STATEMENT AND PRODUCT VISION

2.1 The problem

People often know their current balance and past spending but struggle to understand what will happen to their money in the coming days or months.

They may have upcoming bills, irregular income, recurring payments, unexpected expenses, and several savings goals competing for the same money.

Traditional expense tracking primarily explains where money went. This project should help users understand what might happen next and what actions could improve their financial position.

2.2 The central question

"Given my transaction history, current balance, upcoming obligations, uncertain income, and savings goals, what financial problem might I face next, why might it happen, and what actions could improve the outcome?"

2.3 The solution

Build a privacy-aware, explainable financial decision-support application that:

- Imports and categorizes transactions.
- Detects unusual spending and recurring payments.
- Forecasts future cash flow.
- Identifies possible cash shortages.
- Optimizes savings allocations under explicit constraints.
- Explains recommendations using transaction evidence and calculations.
- Lets users replay historical financial decisions and simulate future alternatives.
- Explains uploaded financial reports side by side with their original content.
- Supports convenient transaction importing and genuine authorized financial-data connections where available.

2.4 Illustrative user scenario

A student receives ₹12,000 monthly. They have a course fee, transport costs, food expenses, recurring payments, and a laptop savings goal.

The student sees ₹8,000 in their current balance and assumes they have enough money.

The application discovers that several payments may occur before the next expected income. It forecasts a possible balance below the student's selected cash buffer.

The system then calculates alternative savings allocations and lets the student test different decisions.

Instead of simply saying "Spend less," the application explains:

- Which upcoming payments contribute to the risk.
- When the balance may fall below the chosen buffer.
- Which assumptions affect the forecast.
- What alternative allocation could improve the situation.
- What trade-off that change creates for the laptop goal.

All values in a real recommendation must be calculated from the application's actual records and assumptions.

3. HACKATHON REQUIREMENTS

The project must address these areas:

Requirement	Expected implementation
Unstructured financial data	Import, normalize, validate, and reconcile transactions
Simulated Open Banking	A clearly labelled mock connector and/or a genuinely authorized integration
Automatic categorization	Rules plus a trained classifier where sufficient data exists
Behavioral anomaly detection	Unusual transaction and recurring-payment alerts
Time-series forecasting	Future daily or weekly balance projections
Shortfall prediction	Identify projected cash-buffer breaches and their dates
Constraint optimization	Allocate available savings across competing goals
Natural-language generation	Explain verified calculations in understandable language
Explainable AI	Show supporting records, formulas, assumptions, and constraints
Privacy preservation	Consent, minimal data retention, access controls, and deletion
Non-fiduciary guidance	Budgeting and cash-flow support without unsupported financial promises
Automatic transaction tracking	Smart Import Hub and honest connection status
Report understanding	Side-by-side report viewer with evidence mapping
Decision simulation	Original-versus-alternative financial trajectories

The scoring priorities are prototype stability, technical depth, innovation, a clear five-minute demo, and understandable UI/UX. Therefore, correctness and end-to-end functionality take priority over unnecessary visual effects or advanced models that cannot be validated.

4. PRODUCT DIFFERENTIATION

Do not build a generic expense tracker with charts and an AI chatbot.

The proposed differentiation is the integration of the following five capabilities into one working decision loop.

A. Advice With Receipts

Every important financial finding and recommendation must show the evidence behind it.

B. Financial Time Machine

Users can replay historical financial decisions and compare hypothetical alternatives without modifying the original transaction ledger.

C. Cash-Flow Stress Tester and Savings Optimizer

Users can change future assumptions, recalculate the forecast, and obtain a feasible savings allocation under explicit constraints.

D. Smart Import Hub and Sync Impact Preview

Users can import transaction data, review changes before applying them, and automatically refresh financial calculations.

E. Side-by-Side Financial Report Explainer

Users can view an original report next to a beginner-friendly explanation, with clickable evidence references and a way to test relevant findings in the Financial Time Machine.

These are the proposed differentiators of this submission. Do not claim that no existing financial product has ever implemented similar functionality.

5. FEATURE A — ADVICE WITH RECEIPTS

Objective

Make every important recommendation traceable to the actual data and calculations that produced it.

Required functionality

For each recommendation, store and display:

- Recommendation ID and title.
- Plain-language explanation.
- Financial problem being addressed.
- Date range analyzed.
- Supporting transaction IDs.
- Supporting report page, section, row, or calculation references, where applicable.
- Formula or method used.
- Forecast version or model identifier.
- Assumptions.
- Constraints affecting the result.
- Estimated benefit and possible trade-off.
- Uncertainty or limitations.
- Whether the result is observed, calculated, predicted, or simulated.

Example

"Your projected balance may fall below your selected ₹1,500 buffer on October 23. This forecast includes the upcoming course fee and rent contribution before your next expected income. Reducing this month's laptop allocation could improve your cash buffer, but may delay your laptop goal."

The actual application must calculate the date, buffer, obligations, and allocation difference. The language model must not invent these values.

Implementation

Create a structured evidence record for each important finding.

A possible schema is:

- `evidence_id`

- `finding_id`
- `source_type`
- `source_id`
- `source_location`
- `calculation_name`
- `calculation_inputs`
- `calculation_output`
- `assumptions`
- `created_at`

Use references to actual database records instead of copying unverified financial values into explanation text.

Use deterministic financial calculations. AI may convert verified outputs into simple language but must not replace the calculation engine.

6. FEATURE B — FINANCIAL TIME MACHINE

Objective

Let users see how a financial decision changes their recorded financial history or projected future.

Historical replay

Display the recorded historical balance trajectory and compare it with a clearly labelled hypothetical alternative.

For example, let the user select an eligible discretionary spending rule and see what the balance might have looked like under an alternative assumption.

Historical counterfactuals must be simulated. Never rewrite the original transaction ledger to make a hypothetical scenario appear real.

Future scenario testing

Allow users to modify:

- A planned discretionary expense.
- An upcoming bill date or amount.
- Expected income date or amount.
- Savings goal amount.
- Goal deadline.
- Minimum cash buffer.
- Recurring-payment assumption.

Run the forecast and savings optimizer again after the scenario changes.

Comparison screen

Show the original plan and modified scenario side by side.

Display:

- Original and simulated balance curves.
- Minimum balance in each scenario.
- Dates when the chosen buffer is breached.
- Estimated difference in available savings.
- Goal progress and target shortfalls.
- Changes in the allocation.
- Which assumptions changed.
- Evidence behind each result.

Use charts with readable axes, clear labels, tooltips, and a visible indication of which curve represents the baseline.

Required safeguards

- Never alter the original ledger during simulation.
- Store scenario assumptions separately.
- Identify every simulated result.
- Explain when a scenario is infeasible.
- Do not present simulated savings as guaranteed savings.
- Keep a reproducible link between a scenario, its assumptions, and its calculation results.

7. FEATURE C — CASH-FLOW FORECASTING AND STRESS TESTING

Objective

Predict future balances and identify potential financial difficulties before they occur.

Forecast equation

Use a transparent baseline:

$$B(t+1) = B(t) + I(t+1) - E(t+1)$$

Where:

- $B(t)$ is the balance at time t .
- $I(t+1)$ is expected income.
- $E(t+1)$ is expected expenditure.

Apply the equation over the selected forecasting horizon.

The system must account for the date and amount of expected transactions, recurring payments, known obligations, and uncertainty in future income and expenditure.

Forecasting horizon

Default to a 30-day forecast, with optional 60-day and 90-day views.

Use daily balances when the data supports them. If the available history is limited, clearly explain the limitations.

Input information

Use:

- Current recorded balance and its date.
- Historical transactions.
- Upcoming known obligations.
- Recurring-payment estimates.
- Expected income.
- User-selected minimum buffer.
- Goal allocations.
- Scenario assumptions.

Allow income to be marked as confirmed, expected, or uncertain.

Historical transactions alone cannot prove that future salary or allowance payments will occur. Treat the certainty level as an explicit input.

Forecast models

Implement the following in order.

Model 1: Transparent baseline

Use known scheduled payments, confirmed or user-entered expected income, and historical spending estimates.

This is the mandatory baseline and should work even when training data is limited.

Model 2: Statistical or ML comparison

When sufficient labelled history exists, compare a rolling-average or seasonal baseline with an appropriate model, such as gradient boosting using lagged spending features.

Potential features include:

- Recent daily and weekly spending.
- Category-level spending averages.
- Day of week.
- Calendar period.
- Recurring-payment indicators.
- Days until expected income.
- Historical balance changes.

Use time-ordered training and testing. Do not randomly mix future and past observations.

Do not train an advanced model solely to claim that the project uses AI. Keep the model only if it improves a relevant evaluation metric.

Model 3: Scenario-based uncertainty

Provide conservative, baseline, and optimistic scenarios.

Allow users to adjust uncertain income and variable spending assumptions.

Only display probabilities when a statistically valid method has been implemented and evaluated. Do not invent percentages or call a scenario probabilistic without justification.

Shortfall detection

Allow users to configure a minimum cash buffer.

Flag projected dates where the balance falls below that buffer.

For each alert, display:

- Date of the projected breach.
- Projected balance.
- Selected buffer.
- Difference between the balance and buffer.
- Contributing transactions or obligations.
- Income assumptions.
- Available alternatives.

Distinguish a projected buffer breach from an actual overdraft or confirmed financial loss.

Forecast evaluation

Use a chronological holdout period.

Calculate mean absolute error:

$$MAE = (1/n) * \text{SUM over } t=1..n \text{ of } | B(t) - \hat{B}(t) |$$

Also measure:

- Shortfall detection recall.
- False-alert rate.
- Lead time before an observed shortfall.
- Performance compared with the baseline.

Display measured results from actual tests, not fabricated model accuracy.

8. FEATURE D — CONSTRAINED SAVINGS OPTIMIZER

Objective

Calculate feasible savings allocations rather than generating generic AI suggestions.

User inputs

For each goal, collect:

- Goal name.
- Target amount.
- Amount already saved.
- Desired deadline.
- Relative priority.
- Minimum required contribution, if applicable.
- Maximum useful contribution in the current period, if applicable.

Also collect the current balance, essential obligations, expected income, and user-selected minimum cash buffer.

Determine available money

Calculate available goal money only after accounting for essential obligations and the protected buffer.

Do not treat the entire current balance as available savings.

Basic mathematical formulation

Let $x(i)$ be the allocation to goal i , and let S be the money available for savings.

$$\text{SUM over } i=1..k \text{ of } x(i) \leq S$$

$$0 \leq x(i) \leq r(i)$$

Here, $r(i)$ is the amount the goal can usefully accept in the current period.

The real multi-period model should account for future cash availability, goal deadlines, target shortfalls, essential obligations, and minimum-balance requirements.

Define an objective that reflects goal priorities and deadline feasibility. Avoid simply maximizing weighted contributions if that produces unrealistic plans.

Use SciPy optimization or Google OR-Tools. Select the simplest solver that reliably handles the required constraints.

Required behavior

The optimizer must:

- Never allocate more money than is available.
- Preserve the configured essential-expense reserve.
- Protect the selected cash buffer.
- Respect goal caps.
- Account for target amounts and deadlines.
- Explain the effect of goal priorities.
- Recalculate when assumptions change.
- Report infeasibility honestly.

If a goal cannot be reached by its deadline, show the gap and possible trade-offs instead of claiming it is feasible.

Optimization explanation

For each allocation, show:

- Recommended contribution.
- Amount still needed.
- Deadline.
- Priority.
- Constraints applied.
- Whether the goal is feasible.

- Which constraints limited the allocation.
- How the allocation changes under another scenario.

Use deterministic solver outputs as the source of truth.

9. FEATURE E — SMART IMPORT HUB AND GOOGLE PAY OPTION

Objective

Make transaction tracking convenient without requiring users to manually enter each expense.

Google Pay connection option

Add a visible "Connect Google Pay" option during onboarding and on the Smart Import Hub page.

When clicked:

1. Explain what data the application intends to access.
2. Check whether a genuine, authorized integration has been configured.
3. If available, launch the real provider-approved connection and consent process.
4. If unavailable, explain that direct Google Pay transaction-history access is not currently supported by the application.
5. Offer CSV, Excel, and supported statement imports.
6. Offer an authorized financial-data provider only if the required integration genuinely exists.

Do not show "Connected" unless a real connection has succeeded.

Do not pretend an uploaded file is a live connection.

Do not scrape Google Pay or private account pages. Never ask for a Google password, bank password, UPI PIN, OTP, or other banking authentication secrets.

Google Pay India APIs are primarily designed for payment acceptance and merchant integration, not general-purpose access to a consumer's complete transaction history. Verify current official documentation before attempting an integration.

Authorized Account Aggregator or financial-data provider

Investigate whether an appropriate authorized provider can be used.

Check:

- Eligibility and onboarding.
- Development or sandbox access.
- Supported institutions and information types.
- Consent and authorization requirements.
- Data refresh behavior.
- Security and retention requirements.
- Feasibility within the hackathon timeline.

If access is unavailable, create a clearly labelled mock connector for demonstration purposes. Never present simulated records as live bank data.

Supported imports

Implement:

- CSV.
- Excel/XLSX.
- Common bank CSV formats.
- PDF statements as an enhancement if reliable extraction is possible.

Data normalization

Convert supported files into a canonical transaction format:

- Transaction ID.
- Source ID.
- Account or dataset ID.
- Transaction date.
- Posting date, when available.
- Original description.
- Normalized merchant description.
- Signed amount.
- Currency.
- Debit or credit direction.
- Category.
- Categorization confidence.
- Transaction reference, when available.
- Running balance, when available.
- Import timestamp.
- Review status.
- Provenance information.

Use one consistent amount convention throughout the application. A recommended convention is positive for credits and negative for debits.

Handle transfers, refunds, duplicate records, malformed dates, missing fields, inconsistent descriptions, and different amount formats.

Transfers between the user's own accounts must not inflate income or expenses.

Import preview

Before committing records, display:

- Number of new transactions.
- Number of duplicate records skipped.
- Records requiring review.
- Invalid records and their rejection reasons.
- Date range.

- Total credits and debits.
- Uncertain categories.

Allow users to review and correct field mappings and categories.

Idempotent imports

Importing the same file repeatedly must not create duplicate records.

Use a stable transaction reference where available. Otherwise, use an appropriate fingerprint and flag ambiguous matches for review.

Do not rely only on date and amount because different transactions can have the same values.

Preserve user corrections. Avoid silently overwriting existing records or deleting transactions that are absent from a partial statement.

Sync Impact Preview

Before committing a new import, show the current and proposed financial situation side by side.

Display:

- New records and skipped duplicates.
- Changes in category totals.
- Changes in recurring-payment estimates.
- Revised minimum projected balance.
- Changes in buffer-breach dates.
- Changes in savings allocations.
- Recommendations that may change.

Explain which newly imported records caused each difference.

Let the user confirm before applying the import.

After confirmation, refresh the relevant financial calculations.

10. FEATURE F — SIDE-BY-SIDE FINANCIAL REPORT EXPLAINER

Objective

Help nontechnical users understand a financial report while preserving the ability to verify every explanation against the original report.

This feature is a major product priority.

Supported formats

Support PDF, CSV, and XLSX reports.

Add scanned-PDF extraction only if it can be implemented reliably.

If parsing fails, show a clear error and offer a supported alternative. Never fabricate extracted figures, report text, page numbers, or tables.

Layout

Create a dedicated page named "Report Explainer."

On desktop:

LEFT PANEL:

- Original report or selected report section.
- Page number, sheet, table, or row reference.
- Highlighted source content when reliable highlighting is possible.

RIGHT PANEL:

- Plain-language explanation.
- Meaning and importance.
- Calculation method.
- Supporting transaction evidence.
- Assumptions.
- Data limitations.
- Classification as reported, calculated, predicted, or simulated.

On mobile, stack the panels vertically and provide simple controls to switch between them.

Click-to-explain

Allow the user to select a supported number, row, table, or section.

Display:

1. What it means in everyday language.
2. Why it matters.
3. The date range covered.
4. The formula or calculation used, if known.
5. Supporting report references.
6. Related transaction IDs, when applicable.
7. Assumptions.
8. Missing information or limitations.
9. Whether the value is reported, calculated, predicted, or simulated.

Example 1

Selected value: Monthly expenses of ₹8,400.

Explain which transactions were included, the date range, category totals, and how transfers or refunds were handled.

Example 2

Selected value: Projected minimum balance of ₹1,200.

Explain the forecast dates, expected income, upcoming expenses, selected buffer, and assumptions that led to the prediction.

Example 3

Selected value: Savings goal progress of 40%.

Show the amount saved, target amount, and calculation used to determine the percentage.

These are illustrative examples. All actual explanations must be generated from real extracted report content or application calculations.

"Where is this covered?" feature

For every important finding or recommendation, show:

- Report name.
- Page, section, table, sheet, or row reference.
- Supporting value or passage.
- Related transaction records.
- Related calculations.
- Missing information.
- What the report does not establish.

For example, a projected cash-buffer shortfall can link to the forecast date, starting balance, upcoming course fee, rent contribution, next expected income, and the relevant assumptions.

Clearly distinguish:

- "The report states this."
- "Our engine calculated this."
- "Our model predicts this."
- "This is a hypothetical scenario."

Never claim that a report covers a subject if there is no supporting content.

Three explanation levels

Provide:

Simple: One or two sentences in everyday language.

Detailed: Supporting records, calculations, assumptions, and caveats.

Technical: Formulas, model outputs, optimization constraints, and data references.

Include a glossary for cash flow, recurring expense, forecast, buffer, anomaly, available savings, and goal allocation.

Use concise language and avoid unexplained financial jargon.

Report comparison

Allow users to compare two reports or two versions of a forecast.

Highlight:

- Changed values.
- Added or missing items.
- Changed spending and income.
- Revised forecast shortfalls.
- Changed savings allocations.
- Differences in assumptions and date ranges.

Warn users when two reports use different date ranges or methodologies.

"Test this finding" action

Allow a relevant finding to open the Financial Time Machine with the finding as a starting point.

For example, a report may identify a projected cash shortage. The user can create a hypothetical scenario involving a reduced or delayed discretionary expense.

Show the original and simulated balance trajectories, buffer breaches, savings allocation changes, and goal trade-offs.

Do not modify the original ledger when a scenario is created.

Explainability safeguards

Financial calculations must be deterministic.

The language model may explain verified calculations but must not invent report content, amounts, percentages, citations, or missing evidence.

If the report is incomplete, ambiguous, or inconsistent, ask the user to review it.

Store the source reference for every important explanation.

11. AUTOMATIC TRANSACTION CATEGORIZATION

Objective

Categorize transactions reliably while allowing users to correct mistakes.

Hybrid pipeline

Stage 1: Normalize

Normalize descriptions, dates, amounts, signs, and currency.

Stage 2: Merchant rules

Use transparent rules for known merchants and recurring descriptions.

Stage 3: ML classifier

When sufficient labelled examples exist, train a text classifier using TF-IDF or word and character n-grams with logistic regression.

Possible categories:

- Income.
- Food and groceries.
- Transport.
- Education and fees.
- Housing.
- Utilities.
- Shopping.
- Healthcare.
- Subscriptions.
- Entertainment.
- Transfers.

- Refunds.
- Other.

Keep the categories configurable.

Stage 4: Confidence and correction

Display a confidence indicator where the model produces a meaningful confidence estimate.

Ask users to review uncertain classifications.

Stage 5: Learn approved mappings

Store user-approved merchant mappings so future imports can reuse corrections consistently.

Do not silently change a user's confirmed category without a clear reason.

Training requirements

Create a reproducible training script, documented dataset schema, preprocessing pipeline, evaluation report, and saved model artifact if ML training is justified.

Do not pretend a model has been trained if no actual training has taken place.

If training data is insufficient, use transparent rules and state that limitation.

12. BEHAVIORAL ANOMALY AND RECURRING-PAYMENT DETECTION

Objective

Detect unusual spending patterns and changes in recurring payments.

Examples:

- A transaction much larger than the user's normal spending in that category.
- A recurring payment increasing unexpectedly.
- A new recurring subscription.
- A sudden rise in weekly food-delivery spending.

Initial anomaly model

Use robust statistical measures.

For a spending category, calculate the historical median m and median absolute deviation:

$$MAD = \text{median}(|x(i) - m|)$$

A possible robust anomaly score is:

$$z_r = 0.6745 * (x - m) / MAD$$

Handle zero or very small MAD values using a suitable fallback rather than dividing by zero.

Use sufficient history before producing user-specific anomaly alerts.

Alert presentation

Display:

- Transaction amount.
- Category.

- Historical comparison baseline.
- Anomaly score or severity where meaningful.
- Reason for the alert.
- Supporting historical transactions.
- Option to confirm that the transaction was expected.

An anomaly does not prove fraud. Use wording such as "Unusual spending detected," not "Fraud detected."

Recurring payments

Estimate recurring patterns using merchant similarity, date intervals, and amount similarity.

Track:

- Merchant.
- Typical amount.
- Expected frequency.
- Last observed payment.
- Next estimated payment.
- Confidence in the recurring pattern.
- Amount changes.

Label inferred future payments as estimates.

13. NATURAL-LANGUAGE EXPLANATION ENGINE

Objective

Translate verified financial outputs into simple, actionable language.

Architecture

Use the following pipeline:

Financial calculations → Structured result → Evidence and assumption references → Explanation template or constrained LLM generation → Validation → UI display.

The financial engine must calculate all amounts, dates, totals, percentages, and allocations.

The LLM must not perform authoritative ledger calculations or invent missing evidence.

Required output

For a financial warning or recommendation, explain:

- What might happen.
- When it might happen.
- Why the engine expects it.
- Which records support the result.
- Which assumptions matter.
- What alternatives could help.
- What trade-offs those alternatives create.

If an external LLM service is used, minimize the information sent to it and disclose the external processing in the Privacy Receipt.

Use a deterministic template-based explanation as a fallback if the LLM is unavailable.

14. DATABASE DESIGN

Use PostgreSQL as the default database for the structured financial records. If the existing project already uses another appropriate database, evaluate it before replacing it.

Use migrations and a consistent data-access layer.

Recommended entities

Users

- `id`
- `email` or chosen authentication identifier, if authentication is implemented
- `created_at`

Financial profiles

- `id`
- `user_id`
- `currency`
- `minimum_buffer`
- `profile_settings`
- `created_at`
- `updated_at`

Accounts and datasets

- `id`
- `user_id`
- `source_type`
- `source_name`
- `connection_status`
- `last_import_at`
- `metadata`

Transactions

- `id`
- `user_id`
- `account_id`
- `source_id`
- `transaction_date`
- `posting_date`

- `description_original`
- `description_normalized`
- `amount`
- `currency`
- `transaction_type`
- `category`
- `category_confidence`
- `external_reference`
- `running_balance`
- `fingerprint`
- `review_status`
- `created_at`
- `updated_at`

Savings goals

- `id`
- `user_id`
- `name`
- `target_amount`
- `saved_amount`
- `deadline`
- `priority`
- `minimum_contribution`
- `maximum_contribution`
- `created_at`
- `updated_at`

Recurring items

- `id`
- `user_id`
- `merchant`
- `typical_amount`
- `frequency`
- `last_observed_at`
- `next_expected_at`
- `confidence`
- `status`

Forecasts

- `id`
- `user_id`
- `dataset_version`
- `model_version`
- `horizon_days`
- `created_at`
- `assumptions`

Forecast points

- `id`
- `forecast_id`
- `forecast_date`
- `predicted_balance`
- `lower_bound`, when valid
- `upper_bound`, when valid
- `minimum_buffer`
- `shortfall_flag`

Scenarios

- `id`
- `user_id`
- `baseline_forecast_id`
- `name`
- `scenario_type`
- `assumptions`
- `created_at`

Scenario results

- `id`
- `scenario_id`
- `minimum_balance`
- `buffer_breach_dates`
- `goal_allocations`
- `goal_shortfalls`
- `result_summary`

Recommendations

- `id`
- `user_id`

- `recommendation_type`
- `title`
- `summary`
- `calculation_result`
- `assumptions`
- `created_at`

Evidence records

- `id`
- `user_id`
- `finding_id`
- `source_type`
- `source_id`
- `source_location`
- `calculation_name`
- `calculation_inputs`
- `calculation_output`
- `created_at`

Imported reports

- `id`
- `user_id`
- `filename`
- `file_type`
- `source`
- `processing_status`
- `created_at`
- `deleted_at`, if soft deletion is required

Report sections

- `id`
- `report_id`
- `page_number`
- `sheet_name`
- `section_title`
- `row_reference`
- `extracted_text`
- `structured_content`

Privacy and import audit records

Record import outcomes, consent-related status changes, deletions, and other relevant data-handling events without unnecessarily logging raw financial information.

Adapt the schema to the actual architecture. Do not create unnecessary tables just to make the design look complicated.

Use foreign keys, appropriate indexes, ownership checks, validation, and database transactions where necessary.

15. FRONTEND REQUIREMENTS

Preferred stack

- Next.js.
- React.
- TypeScript.
- Tailwind CSS.
- Recharts or another suitable charting library.
- A typed API client.
- A consistent component library or reusable design system.

Use the existing project stack if it is already appropriate.

Design direction

The UI must look polished, trustworthy, modern, and easy for a beginner to understand.

Use a clean light theme with white or light-neutral surfaces, dark readable text, restrained blue or teal accents, consistent spacing, and clear typography.

Avoid an overly dark dashboard, excessive neon, glowing effects, crowded cards, unnecessary animations, and generic AI-generated visual decoration.

Prioritize readable charts, simple labels, clear error states, and a consistent layout.

The UI should feel like a real financial planning product, not a collection of unrelated dashboard widgets.

Required pages

1. Landing page.
2. Onboarding and profile setup.
3. Smart Import Hub.
4. Financial overview dashboard.
5. Transaction explorer.
6. Goals and savings optimizer.
7. Cash-flow forecast.
8. Financial Time Machine.
9. Recommendation and evidence detail.
10. Report Explainer.

11.Privacy Receipt and data controls.

12.Settings and supported connection management.

Important UI requirements

- Responsive desktop and mobile layouts.
- Loading states.
- Empty states.
- Clear validation messages.
- Error and retry states.
- Accessible controls and readable text.
- Consistent currency formatting.
- Date-range selectors.
- Useful chart tooltips.
- Clear separation between actual, estimated, forecast, and simulated values.
- No decorative buttons that do nothing.
- No fabricated connection statuses.
- No hardcoded dashboard totals presented as user data.

Use a coherent navigation system and avoid overwhelming users with all features on one screen.

16. BACKEND AND API REQUIREMENTS

Preferred stack

- Python.
- FastAPI.
- Pydantic.
- Pandas and NumPy.
- SQLAlchemy or another suitable database layer.
- Alembic for database migrations when using SQLAlchemy.
- scikit-learn for suitable ML models.
- SciPy optimization or OR-Tools.
- Pytest.

Keep the backend modular and testable.

Suggested API endpoints

Adapt names to the existing architecture.

Health and configuration

- `GET /api/health`
- `GET /api/config/public`

Transactions and importing

- `POST /api/import/preview`
- `POST /api/import/confirm`
- `GET /api/transactions`
- `GET /api/transactions/{id}`
- `PATCH /api/transactions/{id}`
- `DELETE /api/transactions/{id}`
- `GET /api/import/history`

Summary and recurring payments

- `GET /api/finance/summary`
- `GET /api/recurring`
- `GET /api/anomalies`

Forecasting

- `POST /api/forecast`
- `GET /api/forecast/{id}`

Savings optimization

- `GET /api/goals`
- `POST /api/goals`
- `PATCH /api/goals/{id}`
- `DELETE /api/goals/{id}`
- `POST /api/optimize`

Scenarios

- `POST /api/scenarios`
- `GET /api/scenarios`
- `GET /api/scenarios/{id}`
- `POST /api/scenarios/{id}/recalculate`

Recommendations and evidence

- `GET /api/recommendations`
- `GET /api/recommendations/{id}`
- `GET /api/evidence/{id}`

Report explanation

- `POST /api/reports/preview`
- `POST /api/reports/confirm`
- `GET /api/reports`

- GET /api/reports/{id}
- GET /api/reports/{id}/sections
- POST /api/reports/{id}/explain
- GET /api/reports/{id}/coverage
- POST /api/reports/compare
- DELETE /api/reports/{id}

Connection management

- GET /api/connections
- POST /api/connections/{provider}/start
- GET /api/connections/{provider}/status
- POST /api/connections/{provider}/disconnect

Implement connection endpoints only for actual providers or clearly labelled demo adapters. Do not invent a provider API.

Privacy and data management

- GET /api/privacy/receipt
- POST /api/privacy/export
- DELETE /api/privacy/dataset

Use appropriate authentication and authorization for user-specific records. If authentication is implemented, protect every endpoint that accesses personal financial data.

Validate all input and return clear HTTP errors rather than exposing stack traces.

17. CLEAN-CODE AND ENGINEERING STANDARDS

These requirements are mandatory.

Architecture

Separate:

- API routes.
- Request and response schemas.
- Business logic.
- Financial calculations.
- Data ingestion and normalization.
- ML model training and inference.
- Optimization.
- Evidence generation.
- Report parsing.
- Database access.
- Frontend pages.
- Reusable frontend components.

- Tests.

Do not put the entire backend in one large file or the entire frontend in one giant component.

Code quality

- Use meaningful names.
- Add type hints and TypeScript types.
- Keep functions focused on one responsibility.
- Centralize currency and date-handling logic.
- Avoid duplicated business rules.
- Use configuration files and environment variables.
- Validate inputs at system boundaries.
- Handle errors explicitly.
- Document non-obvious calculations.
- Avoid unnecessary abstraction.
- Do not leave essential functionality as TODOs.
- Do not create fake API responses to simulate working features.
- Remove unused imports and dependencies.
- Run formatters, linters, and type checks where configured.

Financial correctness

- Use appropriate decimal arithmetic for monetary amounts, such as Python `Decimal` and a suitable database numeric type.
- Avoid floating-point rounding errors in authoritative monetary calculations.
- Document currency precision and rounding rules.
- Keep one documented signed-amount convention.
- Avoid double-counting transfers and refunds.
- Distinguish transaction date from posting date.
- Reconcile balances when reliable balance information is available.
- Do not infer a current balance from incomplete data without showing the assumption.
- Keep historical records separate from simulated results.

Frontend correctness

- Connect forms to real backend requests.
- Use actual API data for financial figures.
- Show loading, success, empty, and failure states.
- Validate form submissions.
- Make all important actions functional.
- Keep shared state consistent after imports and edits.
- Do not hardcode values that should come from the backend.

- Do not mark operations successful before they actually succeed.

18. PRIVACY AND SECURITY

Financial information is sensitive. Design the application to minimize unnecessary collection and exposure.

Implement the following where applicable:

- User authorization and record ownership checks.
- Secure handling of uploaded files.
- Input validation.
- File type and size restrictions.
- Safe file parsing.
- Protection against unauthorized file access.
- Environment variables for secrets.
- No secrets committed to GitHub.
- No passwords, UPI PINs, or OTPs collected by the application.
- Minimal retention of raw reports and imported data.
- A way to delete stored financial records and uploaded reports.
- Clear consent and connection-status handling.
- No raw financial histories sent to external AI services unless necessary, authorized, and disclosed.
- Appropriate error messages without leaking sensitive data.
- Data isolation between users.
- Tests for unauthorized access and data deletion.

Do not claim production-grade encryption, compliance, zero external transfer, or a certified banking integration unless those claims have been verified.

A hackathon prototype must clearly state any limitations that prevent it from being production-ready.

19. MODEL TRAINING AND REPRODUCIBILITY

If ML models are implemented, provide:

- A documented dataset format.
- A training script.
- A preprocessing pipeline.
- Saved model artifacts where appropriate.
- A prediction interface.
- An evaluation script.
- A fixed random seed where applicable.
- A chronological train/test split for time-series tasks.

- A record of model version and relevant metrics.
- Instructions for reproducing training.

Use synthetic data for development and demonstrations when real user data is unavailable.

Do not claim that the model was trained on real banking data if it was trained on synthetic data.

Do not fabricate accuracy, precision, recall, F1, or MAE scores.

If the dataset is too small for a reliable model, use deterministic rules or a simple baseline and document the limitation.

20. SYNTHETIC DEMO DATA

Provide a synthetic demo dataset that allows the entire application to be demonstrated without requiring a real financial account.

Include realistic examples of:

- Income credits.
- Food and grocery spending.
- Transport.
- Education fees.
- Recurring payments.
- Refunds.
- Transfers between accounts.
- An unusual but legitimate purchase.
- An upcoming expense.
- An uncertain expected income.
- Multiple savings goals.

Include enough history to exercise categorization, recurring detection, forecasting, and scenario testing.

Clearly mark synthetic data as demo data.

Use the demo data to demonstrate:

1. Import preview.
 2. Duplicate detection.
 3. Financial summary.
 4. A forecasted buffer breach.
 5. Evidence-backed recommendations.
 6. A constrained savings allocation.
 7. A Financial Time Machine comparison.
 8. Report explanation and evidence mapping.
 9. Sync Impact Preview.
 10. A data deletion workflow.
-

21. TESTING REQUIREMENTS

Write automated tests for important financial and data-handling logic.

Transaction ingestion tests

- Valid CSV import.
- Valid Excel import.
- Incorrect columns.
- Invalid dates.
- Missing amounts.
- Negative and positive amounts.
- Duplicate records.
- Ambiguous duplicate candidates.
- Overlapping statement ranges.
- Transfers.
- Refunds.
- Reimporting the same file.
- Partial import failure.

Financial calculation tests

- Correct totals.
- Correct signed amounts.
- Starting balance and date handling.
- Transfer exclusion.
- Refund treatment.
- Forecast balance arithmetic.
- Cash-buffer breach detection.
- Missing expected income.
- Different forecast horizons.

ML tests

- Category prediction pipeline.
- Uncertain classification handling.
- User-approved category corrections.
- Anomaly detection on controlled examples.
- Evaluation against a baseline.
- Chronological split for time-series validation.

Optimization tests

- Normal feasible allocation.
- Zero available savings.
- Insufficient funds.
- Goal target already reached.
- Impossible deadline.
- Conflicting constraints.
- Essential expense protection.
- Minimum cash-buffer protection.
- Recalculation after changed assumptions.

Financial Time Machine tests

- Baseline remains unchanged.
- Scenario assumptions are stored separately.
- Forecast recalculation uses modified assumptions.
- Differences are calculated correctly.
- Simulated results are clearly labelled.

Report explainer tests

- PDF, CSV, and Excel parsing where supported.
- Missing or ambiguous values.
- Report page and row references.
- Correct evidence mapping.
- Correct calculation explanations.
- Invalid report handling.
- Report comparison with different date ranges.
- No fabricated citations or page references.

Privacy and integration tests

- Unauthorized record access is blocked.
- Deletion removes or appropriately invalidates stored data.
- Disconnected integrations are not reported as connected.
- No credentials are requested in the import flow.
- Mock provider data is labelled as simulated.
- Import retries do not duplicate records.

Run the available test suite after meaningful changes and fix failures before declaring the application complete.

22. DOCUMENTATION AND DEPLOYMENT

Create a clear `README.md` with:

- Project overview.
- Problem statement.
- Features.
- Architecture.
- Technology stack.
- Repository structure.
- Prerequisites.
- Environment setup.
- Database setup.
- Migration commands.
- Backend run commands.
- Frontend run commands.
- Training commands.
- Test commands.
- Synthetic demo-data instructions.
- API documentation location.
- Environment-variable reference.
- Deployment instructions.
- Security and privacy limitations.
- Current integration limitations.

Create an `.env.example` containing variable names and safe placeholder values, never real credentials.

Include a suitable `.gitignore`.

Document the demo workflow so that a team member can reproduce the presentation from a fresh environment.

If deployment is possible, provide the exact configuration and commands. Do not claim deployment succeeded unless it has been verified.

23. REQUIRED DEVELOPMENT ORDER

Build the application in phases. Do not attempt to polish every page before the core calculations work.

Phase 0 — Repository inspection and plan

Inspect the repository, identify the current stack, list existing functionality, and prepare a concise implementation plan.

Then begin implementation.

Phase 1 — Foundation

Build:

- Database and migrations.
- Transaction schema.
- CSV/XLSX import preview.
- Normalization and validation.
- Duplicate detection.
- Correct ledger calculations.
- Synthetic demo data.
- Financial summary API.
- Basic transaction UI.

Completion condition: A user can import a supported file, review it, save transactions, and see a correct summary.

Phase 2 — Financial intelligence

Build:

- Categorization rules.
- User category corrections.
- Recurring-payment detection.
- Anomaly detection.
- Baseline cash-flow forecasting.
- Buffer-breach detection.
- Evidence records.

Completion condition: A real imported or synthetic dataset produces explainable financial findings.

Phase 3 — Savings optimization

Build:

- Goal management.
- Available-money calculation.
- Constraint solver.
- Goal feasibility checks.
- Allocation explanation.
- Automated tests for constraints.

Completion condition: The optimizer produces feasible allocations or explains why no feasible solution exists.

Phase 4 — Financial Time Machine

Build:

- Scenario creation.
- Assumption editing.
- Baseline and scenario forecasts.

- Original-versus-simulated comparison.
- Goal allocation recalculation.
- Evidence-linked explanations.

Completion condition: Changing an assumption produces a new, testable calculation without altering the original ledger.

Phase 5 — Smart Import Hub

Build:

- CSV/XLSX adapters.
- Additional supported statement formats where feasible.
- Incremental import.
- Deduplication.
- Import history.
- Sync Impact Preview.
- Recalculation after confirmed imports.
- Honest Google Pay connection option.
- Authorized provider integration only if access is available and verified.

Completion condition: The user can review a proposed import, confirm it, and see the financial calculations update correctly.

Phase 6 — Report Explainer

Build:

- Report upload and extraction.
- Side-by-side viewer.
- Click-to-explain interactions.
- Report coverage mapping.
- Evidence links.
- Report comparison.
- "Test this finding" scenario action.
- Error handling for unsupported or ambiguous reports.

Completion condition: The user can verify a finding against the original report and follow its evidence into the financial engine.

Phase 7 — ML evaluation and privacy

Build or finalize:

- Model training scripts where justified.
- Model evaluation.
- Forecast baseline comparison.
- Privacy Receipt.
- Data export and deletion.

- Security checks.
- Automated tests.

Completion condition: Claims about ML, privacy, and integrations match the actual implementation.

Phase 8 — UI polish and demo readiness

Improve:

- Navigation.
- Responsive design.
- Charts.
- Empty and error states.
- Accessibility.
- Loading feedback.
- Report readability.
- End-to-end reliability.
- Documentation.
- Demo workflow.

Completion condition: The entire five-minute demonstration works from start to finish without manually changing database values or pretending that unavailable integrations are connected.

24. FIVE-MINUTE HACKATHON DEMO

Prepare the product around one coherent story.

0:00-0:35 — The problem

Explain that knowing the current balance does not guarantee that money will remain sufficient until the next income.

0:35-1:15 — Import

Use synthetic transaction data or a supported sample file.

Show the import preview, new records, duplicate detection, categories, and updated financial summary.

1:15-2:00 — Prediction and evidence

Show a projected buffer breach.

Open the evidence panel and explain the relevant obligations, income assumptions, and calculations.

2:00-2:45 — Optimization

Show the savings goals and calculated allocation.

Change a goal deadline or upcoming expense and demonstrate a new feasible allocation or an explicit infeasibility explanation.

2:45-3:35 — Financial Time Machine

Compare the baseline with a hypothetical alternative.

Show the balance trajectories, minimum balance, buffer-breach dates, and goal trade-offs.

3:35-4:20 — Report Explainer

Open a sample report.

Select a figure, show its plain-language explanation, and use "Where is this covered?" to reveal the supporting source and calculation.

4:20-4:45 — Automatic tracking

Show the Smart Import Hub and Sync Impact Preview.

Explain that Google Pay connection is enabled only where an authorized integration genuinely exists; otherwise, transaction imports remain available.

4:45-5:00 — Closing

"Our application does more than track past expenses. It predicts potential cash-flow problems, calculates feasible alternatives, and lets users verify the evidence behind every recommendation."

25. NON-NEGOTIABLE DEFINITION OF DONE

Do not declare the project complete until the following conditions are met.

- The frontend and backend run.
 - The database persists records.
 - A user can import and review transactions.
 - Reimporting the same file does not create duplicates.
 - Financial calculations are tested.
 - Transfers and refunds are handled consistently.
 - The cash-flow forecast uses real stored data or clearly labelled demo data.
 - Buffer-breach alerts are calculated.
 - The savings optimizer enforces actual constraints.
 - Recommendations include traceable evidence.
 - Financial Time Machine scenarios do not mutate the original ledger.
 - Report explanations link to real source sections and calculations.
 - Sync Impact Preview shows actual before-and-after changes.
 - Google Pay is never falsely shown as connected.
 - The privacy controls accurately describe implemented behavior.
 - Automated tests run successfully.
 - The README contains usable setup instructions.
 - No essential feature is merely a decorative button or hardcoded result.
 - Known limitations are documented.
-

26. FINAL INSTRUCTIONS TO THE CODING AI

Treat this document as the complete product specification.

Start by inspecting the repository. Then implement the application in the order described above.

Prioritize correctness, clean architecture, financial calculation integrity, evidence traceability, and reliable end-to-end workflows.

Do not rush into advanced ML, PDF OCR, or real banking integration before the transaction pipeline and baseline financial engine work.

Do not fabricate API access, financial data, model evaluation results, report evidence, or successful test runs.

Whenever a feature is incomplete, identify exactly what is missing and continue with the next practical implementation step.

When reporting progress, distinguish:

- Implemented and tested.
- Implemented but not yet tested.
- Partially implemented.
- Not implemented.
- Blocked by an external provider or authorization requirement.

The final result should be a clean, understandable, technically defensible hackathon application that demonstrates financial prediction, constrained optimization, explainability, transaction importing, report understanding, and scenario-based decision support in one integrated product.